Musterlösung zur Übung 4

Primitive Datentypen

Programmierung 1 · WS 2026/27
Bachelor Cyber Security & Privacy · André Kless, M.Sc.

Hinweis: Die Lösungen zeigen einen möglichen Weg zu allen zehn Aufgaben. Andere Programme sind ebenfalls richtig, wenn sie die Vorgaben erfüllen. Es gelten die Eingabevoraussetzungen des Übungsblatts.

Java: Vollständige Programme verwenden Java 25 oder höher und `void main()`. Ergänzende Codeausschnitte stehen jeweils innerhalb dieser Methode. In Rechnungen bezeichnet „Abweichung“ immer Java-Ergebnis minus exakter Wert.

01 Wahrheitswerte und Kurzschluss | Basic

a) Die letzten beiden Spalten sind für jede Belegung gleich. Es gilt das De-Morgan-Gesetz: `!(p && q)` entspricht `!p || !q`.

p	q	!p	p && q	p || q	p ^ q	!(p && q)	!p || !q
false	false	true	false	false	false	true	true
false	true	true	false	true	true	true	true
true	false	false	false	true	true	true	true
true	true	false	true	true	false	false	false

b) Die Klammerung lautet `p || (q && (!p))`. Zuerst bindet `!`, dann `&&`, zuletzt `||`. Für `p = true` und `q = false` ergibt sich `true`. Wegen Kurzschluss bei `||` wird der rechte Operand dabei gar nicht ausgewertet.

c) Mit `&&` erhält `gueltig` den Wert `false`: `anzahl != 0` ist bereits falsch. Deshalb wird `12 / anzahl > 2` nicht ausgewertet und es tritt keine Division durch null auf.

Mit `&` werden beide Operanden ausgewertet. `12 / 0` löst eine `ArithmeticException` aus. Die Initialisierung wird abgebrochen; `gueltig` erhält in dieser Variante keinen Wert. Die beiden Varianten sind getrennt auszuführen.

```
void main() {
    int anzahl = 0;
    boolean gueltig = anzahl != 0 && 12 / anzahl > 2;
    IO.println(gueltig); // false
}
```

Basic: Zahlensysteme und Zweierkomplement

02 Zahlensysteme und Speichergrößen | Basic

a) 173 = 128 + 32 + 8 + 4 + 1, also 10101101_2. Für Oktal: 173 : 8 = 21 Rest 5; 21 : 8 = 2 Rest 5; 2 : 8 = 0 Rest 2. Die Reste rückwärts gelesen ergeben 255_8. Für Hexadezimal: 173 : 16 = 10 Rest 13 (D); 10 : 16 = 0 Rest 10 (A). Ergebnis: AD_{16}.

b) $00110110_2 = 2^5 + 2^4 + 2^2 + 2^1 = 32 + 16 + 4 + 2 = 54$.
$7B_{16} = 7 \cdot 16^1 + 11 \cdot 16^0 = 112 + 11 = 123$.

c) Alle vier Literale bezeichnen denselben Wert. Die Ausgabe erfolgt jeweils dezimal: viermal `173`.

```
void main() {
    int dezimal = 173;
    int dual = 0b10101101;
    int oktal = 0255;
    int hexadezimal = 0xAD;
    IO.println(dezimal);
    IO.println(dual);
    IO.println(oktal);
    IO.println(hexadezimal);
}
```

d) 10 000 · 8 = 80 000 Bit = 10 000 Byte = 10 kB.

03 Zweierkomplement und passende Typen | Basic

a) +37 ist `00100101`. Alle Bits invertieren ergibt `11011010`; plus 1 ergibt `11011011` für −37. Umgekehrt gilt für `11010110`: −128 + 64 + 16 + 4 + 2 = −42.

b) Das Vorzeichenbit wird beim Erweitern wiederholt:

```
−37: 11011011 → 11111111 11011011
+37: 00100101 → 00000000 00100101
```

Führende Einsen bzw. Nullen dürfen in einer verkürzten Darstellung nur entfallen, solange das erste verbleibende Bit weiterhin das richtige Vorzeichen zeigt. Kürzeste Darstellungen hier: −37 als `1011011`, +37 als `0100101` (je sieben Bits). Ein Java-Wert behält trotzdem die feste Bitbreite seines Datentyps.

c) Jeweils kleinster Typ aus den vier vorgegebenen Typen:

Wertebereich	Typ	Begründung
−20 bis 50	`byte`	Liegt innerhalb −128 bis 127.
−20 000 bis 20 000	`short`	Liegt innerhalb −32 768 bis 32 767.
0 bis 50 000	`int`	50 000 überschreitet den Höchstwert von `short`.
0 bis 3 000 000 000	`long`	3 000 000 000 ist größer als 2 147 483 647.

Basic: Rechentyp und Typumwandlung

04 Rechentyp und Typumwandlung | Basic

a) Jede Zeile startet erneut mit `byte a = 90, b = 50;`. „Rechentyp“ bezeichnet hier den Typ, in dem die Addition bzw. Division ausgeführt wird.

Code	Rechentyp	Gespeicherter Wert
`int summe = a + b;`	`int`	`summe = 140`
`byte klein = a + b;`	`int`	`Compilerfehler; kein Wert`
`byte gekuerzt = (byte) (a + b);`	`int`, dann Cast nach `byte`	`gekuerzt = -116`
`double halb = 7 / 2;`	`int`, dann nach `double`	`halb = 3.0`
`double genau = (double) 7 / 2;`	`double`	`genau = 3.5`
`a += b;`	`int`, dann nach `byte`	`a = -116`

Die Addition zweier `byte`-Variablen wird als `int` berechnet. Deshalb ist `byte klein = a + b;` ohne Cast nicht zulässig. Die Werte von `a` und `b` sind hier keine konstanten Ausdrücke; zudem passt die Summe 140 nicht in `byte`.

Bei `7 / 2` findet zuerst Ganzzahldivision statt: Das Ergebnis ist 3. Erst danach wird zu `double` erweitert. Der Cast in `(double) 7 / 2` wirkt dagegen vor der Division und führt zu 3.5.

b) Beim Cast bleiben von 140 nur die unteren acht Bits:

```
140 als int:   00000000 00000000 00000000 10001100
als byte:                                 10001100
Wert:          -128 + 8 + 4 = -116
```

Die anschließende Interpretation als vorzeichenbehaftetes `byte` verändert den Wert. Bei `a += b;` geschieht die Verengung automatisch wie bei `a = (byte) (a + b);`. Der Zieltyp verhindert solche Verluste nicht: Mit einem Cast oder einer Kurzzuweisung kann Information ohne Fehlermeldung verloren gehen.

c) Die Ausgabe lautet `4`. Die einzelnen Schritte sind:

Schritt	Wirkung
1	`a++` liefert als Ausdruck den alten Wert 4.
2	Durch das Inkrement wird `a` vorübergehend auf 5 erhöht.
3	Die Zuweisung speichert den gelieferten Wert 4 wieder in `a`.

Verständlich um eins erhöhen: `a++;` oder `a += 1;`, jeweils ohne zusätzliche Zuweisung an `a`.

```
void main() {
    int a = 4;
    a = a++;
    IO.println(a); // 4
}
```

Java-Sprachspezifikation: Bedeutung des Postfix-Inkrements

Combined: RGB-Farben zerlegen und zusammensetzen

05 RGB-Farben und Bitoperatoren | Combined

a) Die Farbe `0x3366CC` besteht aus folgenden Kanälen:

Kanal	Hexadezimal	Dezimal	Binär mit acht Bits
Rot	33	51	00110011
Grün	66	102	01100110
Blau	CC	204	11001100

Die Verschiebung bringt den gewünschten Kanal in die unteren acht Bits. `& 0xFF` entfernt alle anderen Bits. Beim Zusammensetzen rücken Rot und Grün wieder an ihre Position; `|` verbindet die drei getrennten Bitbereiche.

a–c) Vollständiges Programm für Zerlegung, Zusammensetzen und RGB-Invertierung:

```
void main() {
    int farbe = 0x3366CC;
    int rot = (farbe >> 16) & 0xFF;
    int gruen = (farbe >> 8) & 0xFF;
    int blau = farbe & 0xFF;
    IO.println(rot);   // 51
    IO.println(gruen); // 102
    IO.println(blau);  // 204

    int zusammengesetzt = (rot << 16) | (gruen << 8) | blau;
    IO.println(zusammengesetzt == farbe); // true

    int komplement = ((255 - rot) << 16)
            | ((255 - gruen) << 8) | (255 - blau);
    IO.println(komplement); // 13408563, also 0xCC9933
}
```

c) Neue Kanäle: 255 − 51 = 204 (`CC`), 255 − 102 = 153 (`99`), 255 − 204 = 51 (`33`). Die invertierte Farbe ist somit `0xCC9933`.

Ausgangsfarbe	Invertierte Farbe
0x3366CC	0xCC9933
0x000000 (Schwarz)	0xFFFFFF (Weiß)
0xFFFFFF (Weiß)	0x000000 (Schwarz)
0xFF0000 (Rot)	0x00FFFF (Cyan)

Für die weiteren Tests im Programm nur den Anfangswert von `farbe` ändern. Die Berechnung bleibt gleich.

Combined: Bits invertieren und mit XOR verknüpfen

05 RGB-Farben und Bitoperatoren – Fortsetzung | Combined

d) `~` invertiert alle 32 Bits des `int`. Damit würden auch die oberen acht Bits zu Einsen. Die Maske `0xFFFFFF` setzt sie wieder auf null:

```
int invertiert = (~farbe) & 0xFFFFFF;
IO.println(invertiert == komplement); // true
```

```
farbe:       00000000 00110011 01100110 11001100
~farbe:      11111111 11001100 10011001 00110011
Maske:       00000000 11111111 11111111 11111111
Ergebnis &:  00000000 11001100 10011001 00110011
```

`~farbe` allein ergibt das Bitmuster `0xFFCC9933` und damit einen negativen `int`-Wert. Erst die Maske liefert wieder eine Farbe mit unseren vorgegebenen oberen acht Nullbits.

e) Die folgenden Zeilen können wie d) vor der schließenden Klammer der `main` aus a–c) ergänzt werden:

```
int schluessel = 0xA5C39F;
int geheim = farbe ^ schluessel;
int wiederhergestellt = geheim ^ schluessel;
IO.println(geheim == 0x96A553);       // true
IO.println(wiederhergestellt == farbe); // true
```

```
Farbe:      00110011 01100110 11001100
Schlüssel:  10100101 11000011 10011111
XOR:        10010110 10100101 01010011  = 0x96A553
nochmals ^  10100101 11000011 10011111
Ergebnis:   00110011 01100110 11001100  = 0x3366CC
```

An einer einzelnen Bitposition lässt eine Schlüssel-0 das Bit unverändert; eine Schlüssel-1 kippt es. Zweimal kippen stellt den Ausgangswert wieder her.

Bit	Schlüsselbit	Nach erstem XOR	Nach zweitem XOR
0	0	0	0
0	1	1	0
1	0	1	1
1	1	0	1

Der feste Schlüssel illustriert das Prinzip der Umkehrbarkeit. Er ist keine sichere Verschlüsselung für vertrauliche Daten.

06 Gleitkommazahlen kodieren und vergleichen | Combined

a) 13 = 1101_2; 0,25 = 2^{-2} = $0,01_2$. Daher gilt:

$13{,}25 = 1101{,}01_2 = 1{,}10101_2 \cdot 2^3$

Bestandteil	Ergebnis
Vorzeichen	Positiv, also Vorzeichenbit 0.
Exponent	3 + 127 = 130, binär `10000010`.
Mantissenfeld	`10101000000000000000000` (23 Bits).

Die führende 1 der normalisierten Mantisse wird nicht mitgespeichert. Hinter den fünf benötigten Nachkommabits stehen Nullen. Es ist keine Rundung erforderlich.

```
0 10000010 10101000000000000000000
01000001010101000000000000000000
```

b) Bei `1 10000001 10100000000000000000000` steht die erste 1 für ein negatives Vorzeichen. 10000001_2 = 129; der tatsächliche Exponent ist also 129 − 127 = 2. Die Mantisse lautet $1,101_2$ = 1 + 1/2 + 1/8 = 1,625. Ergebnis: $-1{,}625 \cdot 2^2 = -6{,}5$.

c) `0.1` und `0.2` sind als `double` nicht exakt darstellbar. Das Programm ergibt:

```
void main() {
    double summe = 0.1 + 0.2;
    IO.println(summe);         // 0.30000000000000004
    IO.println(summe == 0.3); // false
    double eps = 1e-12;
    boolean gleichGenug = Math.abs(summe - 0.3) <= eps;
    IO.println(gleichGenug);  // true
}
```

Die absolute Differenz beträgt etwa $5{,}55 \cdot 10^{-17}$ und ist damit kleiner als 10^{-12}. `Math.abs()` macht aus einer gegebenenfalls negativen Differenz ihren nichtnegativen Betrag.

Die Toleranz beschreibt, welche Abweichung im jeweiligen Anwendungsfall akzeptabel ist. Einheit, Größenordnung und geforderte Genauigkeit bestimmen sie. Ein pauschales Epsilon ist nicht für alle Rechnungen geeignet; es korrigiert auch keinen bereits gerundeten Wert.

Combined: Zeichen verschieben

07 Eine Cäsar-Chiffre programmieren | Combined

a) In ASCII hat `'A'` den Code 65 und `'Z'` den Code 90. Für einen Großbuchstaben ergibt `zeichen - 'A'` die Position 0 bis 25. Umgekehrt liefert `(char) ('A' + position)` den Buchstaben. Der Cast ist nötig, weil die Addition zunächst einen `int`-Wert erzeugt.

b) Vollständige Verschlüsselung. Die Eingaben erfüllen die Voraussetzungen des Übungsblatts:

```
void main() {
    String text = IO.readln("Text: ");
    int schluessel = Integer.parseInt(IO.readln("Schlüssel (0-25): "));
    String verschluesselt = "";

    for (int i = 0; i < text.length(); i++) {
        char zeichen = text.charAt(i);
        if (zeichen >= 'A' && zeichen <= 'Z') {
            int position = zeichen - 'A';
            int neuePosition = (position + schluessel) % 26;
            zeichen = (char) ('A' + neuePosition);
        }
        verschluesselt += zeichen;
    }
    IO.println(verschluesselt);
}
```

`% 26` führt nach Z wieder zu A: Bei Z und Schlüssel 3 wird aus der Position 25 zunächst 28 und dann 2, also C. Andere Zeichen werden direkt angehängt.

c) Erwartete verschlüsselte Texte:

Eingabetext	Schlüssel	Verschlüsselter Text
`ABC XYZ! 123`	0	`ABC XYZ! 123`
`ABC XYZ! 123`	3	`DEF ABC! 123`
`ABC XYZ! 123`	25	`ZAB WXY! 123`
`(leerer Text)`	3	`(leerer Text)`
`Abc xyz!`	3	`Dbc xyz!`

Beim leeren Text wird die Schleife kein einziges Mal durchlaufen. Kleinbuchstaben, Leerzeichen, Ziffern und Satzzeichen bleiben unverändert.

Combined: Verschlüsselung rückgängig machen

07 Eine Cäsar-Chiffre – Entschlüsselung | Combined

d) Diese Fortsetzung vor der schließenden Klammer der `main` aus b) einsetzen:

```
String entschluesselt = "";
for (int i = 0; i < verschluesselt.length(); i++) {
    char zeichen = verschluesselt.charAt(i);
    if (zeichen >= 'A' && zeichen <= 'Z') {
        int position = zeichen - 'A';
        int altePosition = (position - schluessel + 26) % 26;
        zeichen = (char) ('A' + altePosition);
    }
    entschluesselt += zeichen;
}
IO.println(entschluesselt);
IO.println(entschluesselt.equals(text)); // true
```

Das zusätzliche `+ 26` verhindert eine negative Zahl vor `% 26`. Beispiel: A hat die Position 0. Beim Rückverschieben um 3 ergibt sich (0 − 3 + 26) % 26 = 23, also X.

Beim Vorwärts- und Rückwärtsverschieben heben sich die Verschiebungen auf. Für alle Tests aus c) entsteht wieder der ursprüngliche Text. Der anschließende Vergleich mit `equals()` liefert jeweils `true`.

`equals()` vergleicht die Textinhalte. == prüft dagegen, ob zwei Variablen dieselbe Referenz enthalten. Zwei getrennt erzeugte Texte können denselben Inhalt haben, obwohl ihre Referenzen verschieden sind. Deshalb ist == kein zuverlässiger Inhaltsvergleich.

Verschlüsselt	Schlüssel	Entschlüsselt
`DEF ABC! 123`	3	`ABC XYZ! 123`
`ZAB WXY! 123`	25	`ABC XYZ! 123`
`Dbc xyz!`	3	`Abc xyz!`

Combined: Zeichenketten und ganzzahlige Geldbeträge

08 Texte und eingelesene Zahlen | Combined

a) Für die Eingabe `Java` lauten die vier Ausgaben:

```
Java!
Java
false
true
```

Zunächst enthalten `text` und `kopie` dieselbe Referenz auf den Text `Java`. `text += "!";` erzeugt einen neuen Text `Java!` und weist dessen Referenz nur `text` zu. `kopie` verweist weiter auf `Java`. String-Werte sind unveränderlich; der ursprüngliche Text wird nicht bearbeitet. Somit ist `text == kopie` falsch, aber `kopie.equals("Java")` wahr.

b) Methoden lassen sich auch unmittelbar am String-Literal aufrufen:

```
IO.println("Programmierung".length());         // 14
IO.println("Programmierung".substring(0, 8)); // Programm
```

`substring(0, 8)` enthält die Zeichen mit den Indizes 0 bis 7. Die obere Grenze 8 gehört nicht mehr zum Teiltext.

c) Der Betrag kann den Wertebereich von `int` überschreiten. Daher werden Einlesen und Rechnen mit `long` durchgeführt:

```
void main() {
    long cent = Long.parseLong(IO.readln("Betrag in Cent: "));
    long euro = cent / 100;
    long restCent = cent % 100;
    IO.println(euro + " Euro und " + restCent + " Cent");
}
```

Eingabe in Cent	Ausgabe
0	0 Euro und 0 Cent
5	0 Euro und 5 Cent
99	0 Euro und 99 Cent
100	1 Euro und 0 Cent
1299	12 Euro und 99 Cent
3000000000	30000000 Euro und 0 Cent

Jeder Cent ist eine ganze Einheit. Die Ganzzahldivision liefert die vollen Euro, `% 100` die übrigen Cent. Beide Ergebnisse sind hier exakt; eine Gleitkommarechnung wäre unnötig.

Challenge: Lotto und verlorene Bits

09 Lotto berechnen und den Überlauf erklären | Challenge

a) Die exakten Ergebnisse der vorgegebenen Formel sind:

Größe	Exakter Wert	Passt in `int`?
Zähler: 49 · 48 · 47 · 46 · 45 · 44	10 068 347 520	Nein
Nenner: 6 · 5 · 4 · 3 · 2 · 1	720	Ja
Anzahl der Tipps	13 983 816	Ja

b) Der erste Überlauf entsteht bei `ersteFuenf * 44`. Das Produkt der ersten fünf Faktoren ist noch 228 826 080 und passt in `int`. Java meldet den Überlauf nicht automatisch. Der Code gibt aus:

```
1478412928
2053351
```

c) Das exakte Produkt benötigt 34 Bits. Rot markiert sind die beiden Bits oberhalb der unteren 32 Bits; blau die gespeicherten Bits:

```
herausfallend  | untere 32 Bits
10             | 01011000 00011110 11001010 10000000
```

Die herausfallende 1 steht an Position 33 und hat den Wert 2^{33} = 8 589 934 592. Das Bit an Position 32 hat den Stellenwert 2^{32} = 4 294 967 296; da es 0 ist, trägt es nichts bei. Die unteren 32 Bits ergeben:

$$01011000\ 00011110\ 11001010\ 10000000_2$$
$$= 0x581ECA80 = 1\ 478\ 412\ 928$$

Das führende der verbleibenden 32 Bits ist 0. Deshalb ist der gespeicherte `int`-Wert hier weiterhin positiv.

d) Die Abweichungen lauten:

Größe	Java minus exakter Wert
Zähler	1 478 412 928 − 10 068 347 520 = −8 589 934 592
Anzahl der Tipps	2 053 351 − 13 983 816 = −11 930 465

Die Zählerabweichung ist genau -2^{33}. Die anschließende Division arbeitet bereits mit dem falschen Zähler und kann die verlorene 1 nicht zurückholen. Zusätzlich liefert Ganzzahldivision nur den ganzzahligen Quotienten: 1 478 412 928 = 2 053 351 · 720 + 208; der Rest 208 wird verworfen.

Challenge: Den Rechentyp rechtzeitig erweitern

09 Lotto berechnen – korrekte Lösung | Challenge

e) Nur `z3` berechnet den Zähler korrekt:

Variante	Gespeicherter `long`-Wert	Warum?
`long z1 = ersteFuenf * 44;`	`1478412928`	Die Multiplikation findet noch in `int` statt.
`long z2 = (long) (ersteFuenf * 44);`	`1478412928`	Der Cast kommt nach dem Überlauf.
`long z3 = (long) ersteFuenf * 44;`	`10068347520`	Der Cast wirkt vor der Multiplikation; gerechnet wird mit `long`.

Ein größerer Zieltyp ändert den Rechentyp der rechten Seite nicht rückwirkend. Mindestens ein Operand muss bereits vor der kritischen Multiplikation den Typ `long` haben.

Vollständige korrekte Variante. Das `L` am ersten Faktor bewirkt schon für die erste Multiplikation eine Rechnung mit `long`:

```
void main() {
    long ersteFuenf = 49L * 48 * 47 * 46 * 45;
    long zaehler = ersteFuenf * 44;
    long nenner = 6L * 5 * 4 * 3 * 2 * 1;
    long anzahl = zaehler / nenner;
    IO.println(zaehler); // 10068347520
    IO.println(nenner);  // 720
    IO.println(anzahl);  // 13983816
}
```

Der größere Wertebereich wird für das Zwischenergebnis im Zähler benötigt. Erst die anschließende Division verkleinert es auf 13 983 816. Es genügt also nicht, nur den erwarteten Endwert mit dem Wertebereich des Datentyps zu vergleichen.

Java-Sprachspezifikation: Ganzzahlmultiplikation und Überlauf

Challenge: Zwei Rechenwege mit Gleitkommazahlen

10 Zwei Rechenwege mit `float` vergleichen | Challenge

a) Mathematisch gilt y + z = 1, also x · (y + z) = 3,5. Auch x · y + x · z = −29 360 128 + 29 360 131,5 = 3,5. Java liefert jedoch:

```
void main() {
    float x = 3.5f;
    float y = -8_388_608f;
    float z = 8_388_609f;
    float geklammert = x * (y + z);
    float ausmultipliziert = x * y + x * z;
    IO.println(geklammert);       // 3.5
    IO.println(ausmultipliziert); // 4.0
}
```

Rechenweg	Java-Ergebnis	Exakt	Abweichung
`x * (y + z)`	3,5	3,5	0
`x * y + x * z`	4,0	3,5	+0,5

b) Die Eingabewerte sind exakt darstellbar. Erst das Produkt `x * z` benötigt mehr signifikante Bits, als `float` speichern kann:

x · z = 3,5 · 8 388 609 = **29 360 131,5**. Im Dualsystem und normalisiert:

$$11100000000000000000000011,1_2$$
$$= 1,11000000000000000000000111_2 \cdot 2^{24}$$

Die führende 1 und die folgenden 23 Bits bilden die 24 verfügbaren signifikanten Bits. Blau markiert ist dieser Teil; die zusätzlichen beiden Bits sind rot:

$$1,11000000000000000000000111_2 \cdot 2^{24}$$

Die führende 1 ist bei normalisierten Zahlen implizit. Gespeichert werden nur die 23 Bits nach dem Komma. Die roten Bits werden bei der Rundungsentscheidung berücksichtigt und anschließend nicht im Mantissenfeld gespeichert.

Challenge: Rundung und Übertrag nachvollziehen

10 Zwei Rechenwege – Rundung im Detail | Challenge

c) Zuerst das Komma wieder um 24 Stellen nach rechts schieben. Dadurch werden die Stellenwerte der roten Bits sichtbar:

$11100000000000000000000011{,}1_2 = 29\ 360\ 131{,}5$

Das erste rote Bit steht jetzt vor dem Komma und zählt 1. Das zweite steht direkt dahinter und zählt 0,5. Zusammen sind es **1,5**. Ohne diese Bits wäre der Wert 29 360 130.

Benachbarter `float`-Wert	Abstand zum exakten Produkt
Unterhalb: 29 360 130	1,5
Oberhalb: 29 360 132	0,5

29 360 132 liegt näher. Deshalb wird aufgerundet: Der beibehaltene Bitteil wird um 1 erhöht. Die letzte blaue Stelle war bereits 1; sie wird zu 0, und der Übertrag erhöht die Stelle davor:

```
  11100000000000000000001
+                       1
= 11100000000000000000010
```

Diese letzte blaue Stelle steht nach dem Zurückschieben des Kommas an der Position 2^1. **+1 im beibehaltenen Bitteil erhöht hier den Zahlenwert um 2**, also von 29 360 130 auf 29 360 132. Die letzten drei Bits wechseln von `001` zu `010`; dabei entsteht der Übertrag.

d) Gerundete Mantisse: $1{,}10000000000000000000010_2$. Exponent: 24 + 127 = 151 = 10010111_2. Das Vorzeichenbit ist 0. Vollständige Kodierung:

```
0 10010111 10000000000000000000010
01001011111000000000000000000010
```

Rundungsabweichung des Produkts: 29 360 132 − 29 360 131,5 = **+0,5**. Die anderen Schritte erklären die Endwerte:

Teilschritt	Wert in Java	Einordnung
`y + z`	1,0	Exakt; anschließend 3,5 · 1 = 3,5.
`x * y`	−29 360 128	Exakt.
`x * z`	29 360 132	Um 0,5 aufgerundet.
`x * y + x * z`	4,0	−29 360 128 + 29 360 132 = 4.

Challenge: Abrunden und mit double vergleichen

10 Zwei Rechenwege – weitere Varianten | Challenge

e) Mit `x = 2.5f` ist das exakte Endergebnis 2,5. Das exakte Produkt `x * z` lautet 2,5 · 8 388 609 = **20 971 522,5**.

Benachbarter `float`-Wert	Abstand zum exakten Produkt
Unterhalb: 20 971 522	0,5
Oberhalb: 20 971 524	1,5

Der untere Wert liegt näher: Das Produkt wird auf 20 971 522 abgerundet. Seine Abweichung beträgt −0,5. `x * y` ist exakt −20 971 520; die Summe der Produkte ergibt daher 2,0.

Rechenweg	Java-Ergebnis	Exakt	Abweichung
`x * (y + z)`	2,5	2,5	0
`x * y + x * z`	2,0	2,5	−0,5

f) Für den ursprünglichen Fall mit x = 3,5 reicht die Genauigkeit von `double` für alle auftretenden Werte aus. Passende Literale tragen hier kein `f`:

```
void main() {
    double x = 3.5;
    double y = -8_388_608.0;
    double z = 8_388_609.0;
    double geklammert = x * (y + z);
    double ausmultipliziert = x * y + x * z;
    IO.println(geklammert);       // 3.5
    IO.println(ausmultipliziert); // 3.5
}
```

`double` besitzt 53 signifikante Bits statt 24 bei `float`. Das hier benötigte exakte Produkt hat nur 26 signifikante Bits. Deshalb geht in diesem Beispiel keine Information verloren.

Ein nachträglicher Cast `(double) ausmultipliziert` im ursprünglichen `float`-Programm ergibt dagegen `4.0`. Er übernimmt nur den schon berechneten Wert in den größeren Datentyp. Die vorangegangene Rundung lässt sich dadurch nicht rückgängig machen.

Die unterschiedlichen Ergebnisse entstehen durch die Rundung der Zwischenergebnisse. Das Distributivgesetz lässt sich deshalb auf endliche Gleitkommarechnungen nicht uneingeschränkt übertragen.

Zur Kontrolle: IEEE-754-Konverter von Harald Schmidt